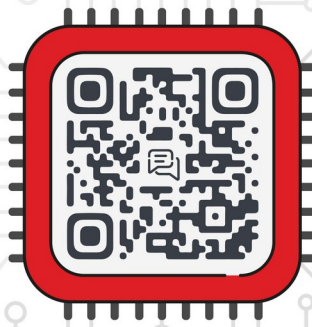




نظري

6



9



1080 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

تتمة البروتوكولات ومقدمة Springboot

د. غسان شذور



محتوى مجاني غير مخصص للبيع التجاري
RB Informatcs; 11/12/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

سنستعرض اليوم في موضوعين أساسيين: تصميم البروتوكولات وكيفية استخدام Spring Boot لتبسيط عملية بناء التطبيقات. مع التركيز على الأمثلة العملية لضمان الفهم الكامل للمفاهيم.

تصميم البروتوكول لتطبيقات الشبكات

تصميم البروتوكول هو عملية تحديد القواعد والصيغ التي تتبعها التطبيقات للتواصل عبر الشبكات. يمكننا تشبيهه بإنشاء "لغة" يفهمها كل من العميل والخادم.

يحدد البروتوكول ما يلي:

- صيغة الرسالة - كيف يتم تنظيم البيانات
- تسلسل الرسائل - ترتيب التواصل (ترتيب تبادل الرسائل)
- التعامل مع الأخطاء - ماذا نفعل عندما تحدث مشكلة
- التحقق من الهوية - كيف نعرف هوية الأطراف المتواصلة
- أنواع البيانات - ما نوع البيانات التي يمكن إرسالها

Protocol Patterns - Request/Response

النمط الأكثر شيوعاً: طلب العميل، استجابة الخادم

الخادم → العميل: استجابة

العميل → الخادم: طلب

أمثلة:

1. HTTP: GET /index.html → 200 OK + Content
2. من قاعدة البيانات: يتم تنفيذ استعلام على جدول المستخدمين لاستخراج البيانات المطلوبة.
3. من خلال RPC: يتم استدعاء الدالة "login" لإرجاع كائن المستخدم الذي يحتوي على بيانات تسجيل الدخول.
4. الخصائص: تزامنية، رسائل مترابطة

ملاحظة: نمط الطلب والاستجابة هو الأكثر شيوعاً، ويشبه في فكرته عملية طرح سؤال والحصول على إجابة. يعد بروتوكول HTTP المثال الكلاسيكي على هذا النمط، إذ يعمل بكفاءة في الحالات التي تتطلب استجابة محددة لمتابعة التنفيذ.

Protocol Patterns - Publish/Subscribe

التواصل المدفوع بالأحداث (Event-Driven Communication)

الناشر → الوسيط → المشتركين: توزيع الحدث

الناشر → الوسيط: إرسال حدث

أمثلة:

1. غرف الدردشة: رسائل المستخدم تُرسل إلى جميع أعضاء الغرفة.
2. أسعار الأسهم: تحديثات الأسعار تُبث لجميع المشتركين.
3. الإشعارات: أحداث النظام تُرسل إلى العملاء المهتمين.
4. الخصائص: غير متزامن (Asynchronous), One-to-Many

ملاحظة: نظام النشر والاشتراك يُستخدم في الأنظمة المبنية على الأحداث، حيث يرسل الناشر أحداثاً يتلقاها المستمعون المهتمون دون تواصل مباشر بينهم. يشبه ذلك غرف الدردشة أو شاشات الأسهم، ويمتاز بقدرته على البث الواسع وفصل مكونات النظام لزيادة المرونة وقابلية التوسع.

Designing Message Headers

الـ Headers الشائعة:

الوصف	الحقل
إصدار البروتوكول (1 بايت)	الإصدار
نوع الرسالة (1 بايت، مثل 1=نص، 2=انضمام، 3=مغادرة)	النوع
حجم الحمولة (2 بايت)	الطول
ترتيب الرسائل (4 بايت)	التسلسل
سلامة البيانات (2 بايت)	مجموع التحقق

ملاحظة: الـ Headers هي البيانات الوصفية التي ترافق البيانات الفعلية في الرسائل. الـ Header الجيد يحتوي على معلومات كافية لتوجيه الرسالة ومعالجتها دون الحاجة لقراءة محتواها الداخلي. تستخدم الأرقام المميزة للتعرف على الرسائل الصحيحة، وتساعد أرقام التسلسل في تتبع الترتيب، بينما تضمن عمليات التحقق من الصحة سلامة البيانات أثناء النقل.

Error Handling in Protocols

امثلة على انواع الاخطاء:

1. أخطاء على مستوى التطبيق.
2. أخطاء في الشبكة (يتعامل معها بروتوكول TCP).
3. أخطاء في البروتوكول (يتعامل معها التطبيق).
4. أخطاء في منطق العمل.

مثال على صيغة الاستجابة عند حدوث خطأ:

```
"error_code": "INVALID_MESSAGE",
"description": "نوع الرسالة غير معروف",
reference_id": 12345"
```

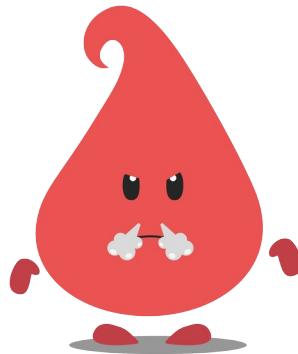
ملاحظة: الشبكات ليست دائماً موثوقة، لذا يجب أن تتعامل البروتوكولات مع الأخطاء بمرونة وكفاءة. ورغم أن بروتوكول TCP يعالج الأخطاء على مستوى الشبكة، فإن بروتوكولات التطبيقات تحتاج آلياتها الخاصة لذلك. أما الاستجابات الجيدة للأخطاء، فيجب أن توضح للعميل سبب المشكلة وكيفية تصحيحها إن أمكن.

State Management

مقارنة إدارة حالة البروتوكول

البروتوكولات ذات الحالة	البروتوكولات عديمة الحالة
المعلومات	كل طلب يحتوي على جميع المعلومات اللازمة
المثال	الخادم يحتفظ بحالة الاتصال
الميزة	HTTP (مع الكوكيز)
العيب	أداء أفضل
	استخدام موارد الخادم
	قابلية التوسع
	تكرار بيانات المصادقة

ملاحظة: قرار تصميم مهم هو ما إذا كان البروتوكول سيكون بحالة أم بدون حالة. البروتوكولات بدون حالة، مثل HTTP، تتعامل مع كل طلب بشكل مستقل مما يسهل التوسع، بينما البروتوكولات بحالة تحتفظ بالسياق بين الرسائل، وهو ما يجعلها أنسب للمحادثات الطويلة. يعتمد الاختيار على متطلبات التطبيق.



Security Considerations

الجوانب الأمنية للبروتوكول:

1. المصادقة: من هو المرسل؟
2. التفويض: ماذا يمكنه أن يفعل؟
3. السلامة: هل تم العبث بالرسالة؟
4. السرية: هل المحتوى خاص؟

الطرق الشائعة:

استخدام TLS لأمان النقل.

رموز JWT للمصادقة.

التوقيعات لضمان سلامة الرسائل.

ملاحظة: الأمن ليس إضافة في نهاية التطوير، بل يجب أن يكون جزءاً من تصميم البروتوكول منذ البداية، مع مراعاة التحقق من الهوية، والتفويض، وسلامة الرسائل، والسرية. يتولى بروتوكول TLS تأمين النقل، بينما يحدد تصميم البروتوكول الجوانب الأمنية الداخلية. أما رمز الويب JSON أو JWT، فهو معيار لنقل المعلومات بأمان بين الأطراف في شكل JSON، ويُستخدم على نطاق واسع في التحقق من الهوية وتبادل البيانات في تطبيقات الويب.

HTTP/1.1

الطلب والرد المعتمدان على النص يُظهران طريقة تواصل بسيطة وواضحة بين العميل والخادم.

- الطلب:

GET /index.html HTTP/1.1

Host: example.com

User-Agent: Mozilla/5.0

- الرد:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 1234

- الخصائص:

1. لا يحتفظ بحالة.
2. يمكن قراءته بسهولة من قبل البشر.
3. يحتوي على تفاصيل كثيرة.

ملاحظة: يظهر بروتوكول HTTP كيف يمكن لبروتوكول نصي أن يدعم شبكة الويب بكاملها. سهّلت قابليته للقراءة تصحيح الأخطاء واعتماده الواسع، لكن كثرة التفاصيل فيه سببت مشاكل أداء عالجتها الإصدارات اللاحقة. يوضح ذلك كيف تؤثر قرارات التصميم المبكرة على الأنظمة في المدى الطويل.

Protocol Evolution

التصميم من أجل التغيير:

- **التفاوض على الإصدار:** يتفق العميل والخادم على الإصدار المستخدم.
- **الحقول الاختيارية:** السماح بإضافة حقول جديدة دون التأثير على العملاء القدامى.
- **التوافق العكسي:** تمكين العملاء القدامى من العمل مع الخوادم الجديدة دون مشاكل.
- **استراتيجية الإيقاف التدريجي:** إزالة الميزات تدريجيًا وبأسلوب منظم.

مثال: رأس إصدار بروتوكول HTTP.

ملاحظة: البروتوكولات الناجحة تستمر طويلاً، لذا يجب أن تتطور دون الإضرار بالأنظمة القائمة. يتيح التفاوض على الإصدارات والحقول الاختيارية والتغييرات التدريجية نموها بسلاسة. يساعد التصميم المرن منذ البداية على تجنب المشكلات المستقبلية.

Performance Considerations

تحسين أداء البروتوكول:

- **حجم الرسالة:** كلما كانت الرسالة أصغر، كان استهلاك النطاق الترددي أقل.
- **جولات الاتصال:** تقليل عدد الجولات يقلل زمن التأخير.
- **عبء التحليل:** يعتمد على ما إذا كانت الرسائل بصيغة ثنائية أم نصية.
- **الضغط:** يفيد بشكل خاص مع البيانات الكبيرة.
- **التسلسل المتعدد:** يتيح إرسال عدة طلبات دون انتظار الرد على كل واحد منها.

ملاحظة: تصميم البروتوكول يتطلب موازنة بين عوامل تؤثر على الأداء. الرسائل الصغيرة تقلل استهلاك النطاق الترددي لكنها قد تتطلب المزيد من التبادلات، بينما التسيقات الثنائية أسرع في التحليل لكنها أصعب في تصحيح الأخطاء. يعتمد الاختيار على طبيعة التطبيق والقيود المحيطة به.

Best Practices

- اختر صيغة التسلسل المناسبة.
- صمم إطار الرسائل بوضوح.
- حدد أنواع الرسائل وصيغها بدقة.
- خطط لكيفية التعامل مع الأخطاء.
- فكر في إدارة الحالة ضمن النظام.
- عالج متطلبات الأمان الأساسية.
- خطط للإصدار والتطوير المستقبلي.
- وثّق مواصفات البروتوكول بشكل شامل.

Spring Boot

ما قمنا ببنائه في المحاضرات السابقة كان:

1. خوادم تطبيقات جافا باستخدام المقابس (Websockets).

2. تحليل الطلبات مثل (HTTP) يدويًا.

3. بناء الاستجابات مثل (HTTP) يدويًا.

4. إدارة الخيوط لعمالء متعددين.

ولكن...

يمكن أن يكون هناك الكثير من الأسطر لتطبيقات بسيطة أساسية.

لا توجد طريقة سهلة لإضافة ميزات (المصادقة، قواعد البيانات، ..).

يصعب الحفاظ عليه مع نموه.

التحدي في العالم الحقيقي (Challenge World-Real The)

يحتاج بناء تطبيق حديث إلى:

التعامل مع الالف الطلبات/الثانية.

الاتصال بقواعد البيانات،

إرسال رسائل البريد الإلكتروني،

مصادقة المستخدم والأمان.

طريقة منظمة للتعامل مع عناوين URL المختلفة.

معالجة الأخطاء والتسجيل (Logging).

تحويل بيانات XML/JSON.

إدارة التهيئة والمراقبة والمقاييس....

لماذا نعيد اختراع العجلة في كل مرة ؟

نحن بحاجة إلى مصادقة آمنة. نحن بحاجة إلى التحدث إلى قواعد البيانات، وإرسال رسائل البريد الإلكتروني، ومعالجة الملفات، وإنشاء التقارير. نحن بحاجة إلى طرق منظمة للتعامل مع عناوين URL المختلفة. نحن بحاجة إلى معالجة أخطاء

متسقة. نحن بحاجة إلى تهيئة إعدادات مختلفة للتطوير مقابل الإنتاج. إذا قمنا ببناء كل هذا من

المقابس (Websockets) في كل مرة، فلن ننتهي أبدًا من شحن أي شيء: لقد حلت الصناعة هذا الأمر منذ عقود: نحن

نبني على أطر العمل (frameworks).

Frameworks

يتولى إطار العمل البنية التحتية المملة والمتكررة. أنت تكتب منطق العمل الفريد الذي يجعل تطبيقك مميزًا .

ما هو إطار العمل؟

1. هيكل معد مسبقًا للتطبيقات.
 2. يوفر حلولاً شائعة للمشاكل الشائعة.
 3. أنت تملأ منطق العمل الخاص بك.
- تشبيه: بناء سيارة من الصفر مقابل استخدام هيكل (chassis) بمحرك مثبت مسبقًا. أنت تركز على التصميم/الداخل، وليس على البراغي والمكابس.
- تتولى أطر العمل البنية التحتية المملة والمتكررة.

Why Spring Boot Specifically?

حوالي 60% من تطبيقات جافا للمؤسسات تستخدم Spring. تستخدمه شركات مثل Netflix، Amazon، Airbnb، Uber.

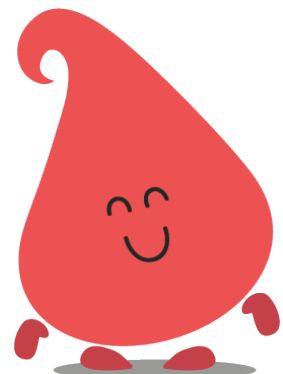
SpringBoot جعل Spring أسهل :

قبل: الكثير من ملفات XML للتهيئة.

الآن: صفر XML.

خادم ويب كامل في 6 أسطر من الكود.

```
@Controller
public class GreetingController {
    @GetMapping("/greet")
    @ResponseBody
    public String greet() {
        return "Hello from Spring!";
    }
}
```



What Problem Does Spring Boot Solve?

إدارة المنافذ، إنشاء مجموعات الخيوط، تحليل طلبات العميل (مثل HTTP)، التحويل إلى JSON - كل هذا عمل يدوي. يوفر Spring Boot : خادمًا مدمجًا حتى لا نضطر إلى تثبيت Tomcat بشكل منفصل، وتهيئة تلقائية حتى لا نكتب كودًا روتينيًا، ودعم JSON مدمج حتى لا نقوم بالتحليل يدويًا، وميزات إنتاج مثل فحوصات الصحة (Health Checks) من اليوم الأول. إنه لا يفعل أي شيء لا يمكننا القيام به بأنفسنا - إنه فقط يقوم بكل العمل المتكرر نيابة عنا.

سحر الخادم المدمج :

قبل:

تثبيت Tomcat بشكل منفصل.

نشر ملفات WAR.

تهيئة server.xml.

الآن:

مع Spring Boot، يأتي الخادم مدمجًا داخل تطبيقك.

تقوم بتشغيله مثل أي برنامج جافا: `java -jar myapp.jar`.

لا حاجة لتثبيت خادم خارجي.

ملفات JAR : تستخدم عند توزيع برامج جافا التي لا تحتاج إلى التشغيل على خادم ويب.
ملفات WAR: مثالية لتطبيقات الويب التي تتطلب خادم ويب أو خادم تطبيقات للتشغيل.

Auto-Configuration: How It Works

مبدأ "العُرف فوق التهيئة" (Convenient over Configuration)

مبدأ العُرف فوق التهيئة يعني أن الإطار (Framework) يعتمد على إعدادات افتراضية ذكية مستندة إلى العُرف (Convention)، بحيث لا يحتاج المطور إلى كتابة إعدادات (Configuration) كثيرة إلا في الحالات الخاصة أو عندما يريد سلوكًا مختلفًا عن الافتراضي. بمعنى آخر، إذا التزمَ بالعُرف الذي يتوقعه الإطار، فسيتم إعداد كل شيء تلقائيًا دون الحاجة إلى تهيئة إضافية.

مثال: اتصال قاعدة البيانات

• تضيف تبعية MySQL:

يقوم Spring Boot باكتشاف وجود التبعية (Dependency) الخاصة بـ MySQL في المشروع، ويقوم تلقائيًا بتهيئة مجمّع الاتصال (Connection Pool) مع قاعدة البيانات دون أي إعداد يدوي من المطور.

• لا تضيف MySQL:

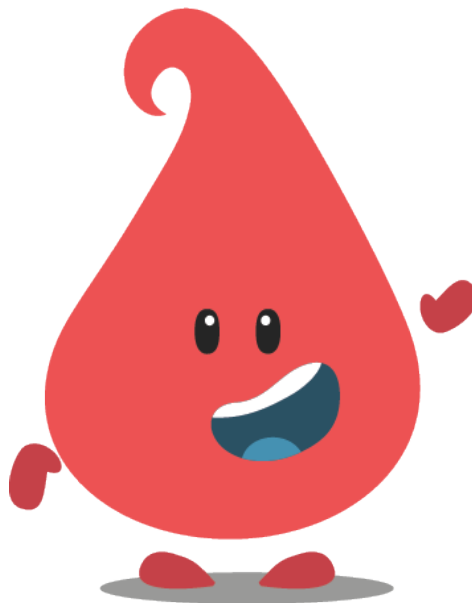
في حال عدم إضافة تبعية MySQL، فإن Spring Boot لن يتهياً لإنشاء اتصال بقاعدة البيانات، لأنه يعتمد على العُرف القائل بأن وجود التبعية دليل على نية المطور استخدام قاعدة بيانات.

مثال برمجي توضيحي

```
if (classpath.has(DataSource.class)) {  
    setupDatabaseConnection();  
}
```

:classpath.has(DataSource.class)

تتحقق هذه الدالة مما إذا كانت فئة DataSource متاحة في المسار الحالي للفئات (Classpath). إذا كانت الفئة موجودة، فهذا يعني أن النظام جاهز للتعامل مع اتصالات قاعدة البيانات، فيتم عندها استدعاء الوظيفة setupDatabaseConnection() لإعداد الاتصال.



THE END